

Regresión Lineal, polinómica y múltiple

Tres intentos desesperados de domar un diamante con fórmulas que terminan tan rotas como su modelo

Daniel Barajas Higuera

MLEARN



Contenido

- 1 Dataset Diamonds
- 2 Regresión Lineal
- 3 Regresión Polinómica
- 4 Regresión Múltiple

Dataset Diamonds

- Conjunto de datos con características físicas y de calidad de diamantes.
- **Variables principales:**
 - carat: Peso del diamante.
 - price: Precio del diamante (valor objetivo).
 - cut, color, clarity: Descripciones de calidad.
- Pregunta clave: ¿Cómo predicen las características el precio del diamante?



Carga del Dataset

```
diamonds = sns.load_dataset('diamonds')

outliersx = (diamonds['x'] < 3) | (diamonds['x'] > 20)
outliersy = (diamonds['y'] < 3) | (diamonds['y'] > 20)
outliersz = (diamonds['z'] < 1) | (diamonds['z'] > 20)

unusual = outliersx | outliersy | outliersz

diamonds[unusual][['x', 'y', 'z', 'price']].head()

diamonds[unusual].shape

# Eliminamos datos atípicos
diamonds = diamonds[~unusual].copy()

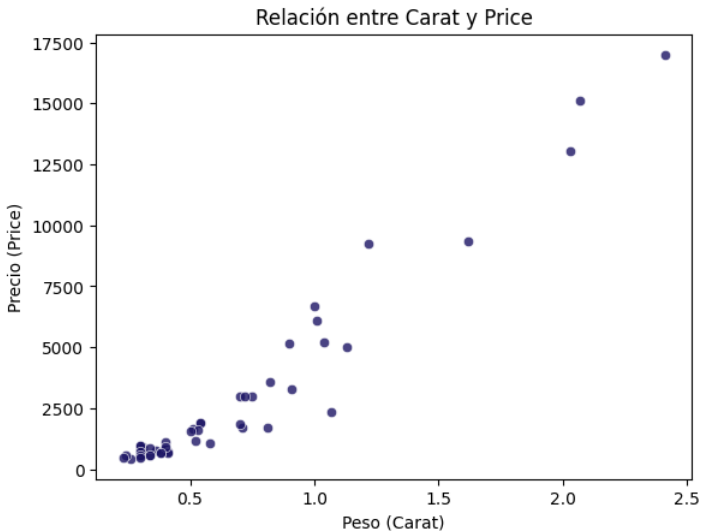
muestra = diamonds.sample(50, random_state=42)
```

Visualización de Datos

- Relación entre carat y price: Los precios aumentan con el peso del diamante.

```
sns.scatterplot(x='carat', y='price', data=diamonds, alpha=0.5)
plt.title('Relación entre Carat y Price')
plt.xlabel('Peso (Carat)')
plt.ylabel('Precio (Price)')
plt.show()
```

Visualización de Datos



Regresión



Contenido

- 1 Dataset Diamonds
- 2 Regresión Lineal**
- 3 Regresión Polinómica
- 4 Regresión Múltiple

Modelo de Regresión Lineal Simple

- **Ecuación:** $\hat{y} = \theta_0 + \theta_1 x$
- **Definiciones:**
 - $\hat{y}$: Valor predicho (e.g., price).
 - x : Variable independiente (e.g., carat).
 - θ_0 : Intersección.
 - θ_1 : Pendiente.
- **Objetivo:** Estimar θ_0 y θ_1 minimizando el error entre valores predichos y reales.

Entrenamiento del Modelo Simple

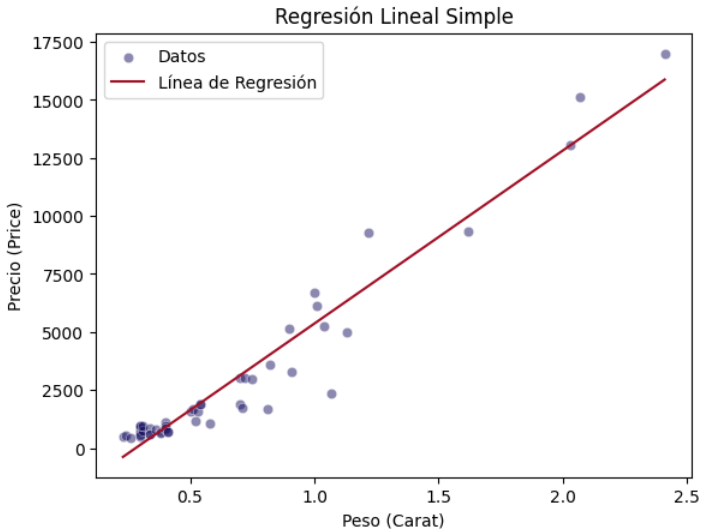
```
from sklearn.linear_model import LinearRegression
datos = muestra.copy()
X = datos[['carat']]
y = datos['price']
model = LinearRegression()
model.fit(X, y)
```

Visualización del Modelo de Regresión Simple

- Línea ajustada que muestra la relación promedio entre carat y price.

```
datos['predictions'] = model.predict(X)
sns.scatterplot(x='carat', y='price', data=datos, alpha=0.5, label='Datos')
sns.lineplot(datos, x='carat', y='predictions',
              color=est_palette[3], label='Línea de Regresión')
plt.title('Regresión Lineal Simple')
plt.xlabel('Peso (Carat)')
plt.ylabel('Precio (Price)')
plt.legend()
plt.show()
```

Visualización del Modelo de Regresión Simple



Propiedades Relevantes del Modelo LinearRegression

- En una regresión lineal simple, las siguientes propiedades de `model` son clave para interpretar el modelo:

```
print(f"Coeficiente: {model.coef_[0]}")  
print(f"Intersección: {model.intercept_}")  
print(f"R^2: {model.score(X, y)}")
```

- `coef_`:
 - Representa el coeficiente (θ_1 , pendiente de la línea).
 - Indica cuánto cambia `price` por cada unidad adicional de `carat`.
- `intercept_`:
 - Es la intersección (θ_0), el valor de `price` cuando `carat` es 0.
- `score(X, y)`:
 - Devuelve el R^2 , que mide qué tan bien los valores predichos explican los valores reales.
 - Un R^2 cercano a 1 indica un ajuste fuerte.

Ejemplo de interpretación

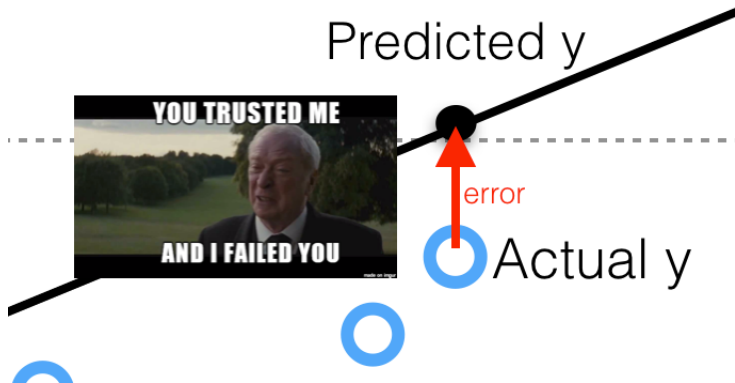
$$\hat{y} = \text{intercept_} + \text{coef_}[0] \cdot x$$

Ejemplo: Si $\text{coef_} = 5000$ y $\text{intercept_} = 1000$:

$$\hat{y} = 1000 + 5000 \cdot x$$

Esto significa que por cada incremento de 1 en `carat`, `price` aumenta en 5000.

Métricas



Métricas

```
datos_metricas = datos[['carat', 'price', 'predictions']].copy()
# Calcular errores
datos_metricas['error'] = datos_metricas['price'] - datos_metricas['predictions']
datos_metricas['error_abs'] = datos_metricas['error'].abs()
datos_metricas['error_squared'] = datos_metricas['error'] ** 2
datos_metricas.head(10)
```

	carat	price	predictions	error	errorabs	errorsquared
37598	0.230000	485	-378.904470	863.904470	863.904470	746330.933370
1390	0.240000	559	-304.356999	863.356999	863.356999	745385.308306
28629	0.260000	434	-155.262058	589.262058	589.262058	347229.772901
33878	0.300000	844	142.927825	701.072175	701.072175	491502.194665
41259	0.300000	501	142.927825	358.072175	358.072175	128215.682563
9716	0.300000	590	142.927825	447.072175	447.072175	199873.529726
29848	0.300000	710	142.927825	567.072175	567.072175	321570.851744
45312	0.300000	524	142.927825	381.072175	381.072175	145216.002617
36811	0.300000	956	142.927825	813.072175	813.072175	661086.361881
36777	0.300000	956	142.927825	813.072175	813.072175	661086.361881

Métricas

```
# Métricas
mse = datos_metricas['error_squared'].mean()
rmse = mse ** 0.5
mae = datos_metricas['error_abs'].mean()
rae = datos_metricas['error_abs'].sum() / datos_metricas['price'].sum()

print(f"MSE (Mean Squared Error): {mse}")
print(f"RMSE (Root Mean Squared Error): {rmse}")
print(f"MAE (Mean Absolute Error): {mae}")
print(f"RAE (Relative Absolute Error): {rae}")
```

- **MSE (Mean Squared Error):**

- Promedio de los errores al cuadrado.
- Penaliza errores grandes más que los pequeños.

$$MSE = \frac{1}{n} \sum_{i=1}^n (\text{price}_i - \text{predictions}_i)^2$$

- **RMSE (Root Mean Squared Error):**

- Raíz cuadrada del MSE.
- Útil para interpretar el error en las mismas unidades que la variable dependiente.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\text{price}_i - \text{predictions}_i)^2}$$

- **MAE (Mean Absolute Error):**

- Promedio de los valores absolutos de los errores.
- Más robusto frente a valores atípicos.

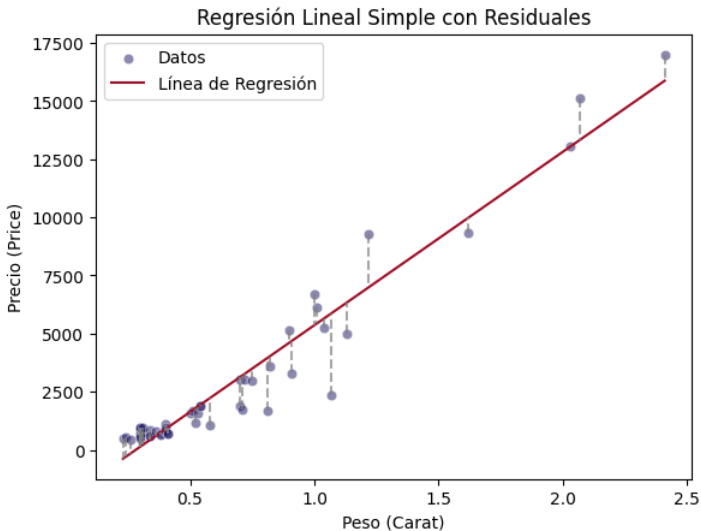
$$MAE = \frac{1}{n} \sum_{i=1}^n |\text{price}_i - \text{predictions}_i|$$

- **RAE (Relative Absolute Error):**

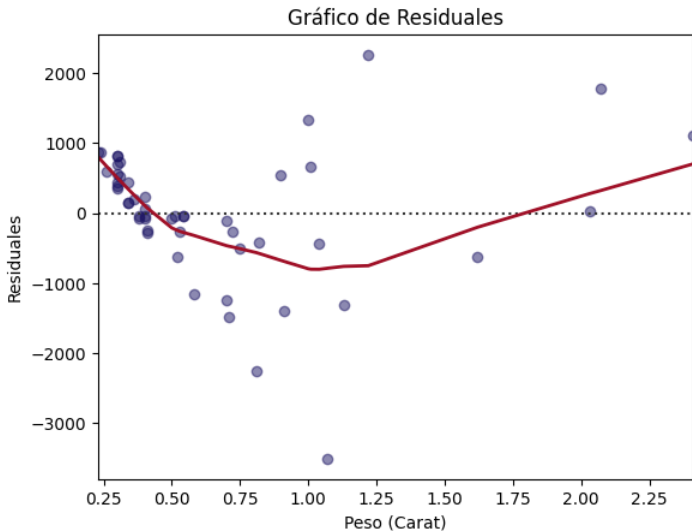
- Relación entre la suma de errores absolutos y la suma de errores respecto al promedio.

$$RAE = \frac{\sum_{i=1}^n |\text{price}_i - \text{predictions}_i|}{\sum_{i=1}^n |\text{price}_i - \bar{\text{price}}|}$$

Residuales



Residuales



Métricas

```
rss = datos_metricas['error_squared'].sum()
tss = ((datos_metricas['price'] - datos_metricas['price'].mean()) ** 2).sum()
r2 = 1 - (rss / tss)

print(f"RSS (Residual Sum of Squares): {rss}")
print(f"TSS (Total Sum of Squares): {tss}")
print(f"R2 (Coeficiente de Determinación): {r2}")
```

Métricas: R^2

- **RSS (Residual Sum of Squares):** Suma de los errores al cuadrado, mide la variabilidad no explicada por el modelo.

$$RSS = \sum_{i=1}^n (\text{price}_i - \text{predictions}_i)^2$$

- **TSS (Total Sum of Squares):** Variabilidad total de la variable dependiente respecto a su media.

$$TSS = \sum_{i=1}^n (\text{price}_i - \bar{\text{price}})^2$$

- **R^2 (Coeficiente de Determinación):**
 - Proporción de la variabilidad explicada por el modelo.
 - Mide qué tan bien los valores predichos se ajustan a los valores reales.
 - Valores cercanos a 1 indican un modelo con buen ajuste.

$$R^2 = 1 - \frac{RSS}{TSS}$$

Métricas

```
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.metrics import mean_absolute_error, root_mean_squared_error

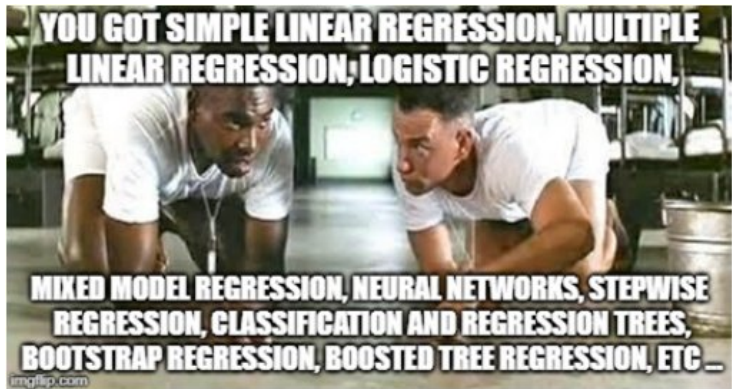
# Calcular métricas
r2 = r2_score(datos['price'], datos['predictions'])
mse = mean_squared_error(datos['price'], datos['predictions'])
rmse = root_mean_squared_error(datos['price'], datos['predictions'])
mae = mean_absolute_error(datos['price'], datos['predictions'])

# Mostrar resultados
print(f"R2 (Coeficiente de Determinación): {r2}")
print(f"MSE (Mean Squared Error): {mse}")
print(f"RMSE (Root Mean Squared Error): {rmse}")
print(f"MAE (Mean Absolute Error): {mae}")
```


Interpretación de los Resultados

- **MSE:** Indica el error cuadrático medio. Un valor más pequeño sugiere menos errores grandes.
- **MAE:** Muestra el error absoluto medio, indicando el promedio de las desviaciones en valor absoluto entre las predicciones y los valores reales.
- **RMSE:** Refleja el tamaño del error en las mismas unidades de la variable objetivo.
- **R²:** Muestra la proporción de la variabilidad en el índice de desempeño explicada por el modelo.

Regresión



Contenido

- 1 Dataset Diamonds
- 2 Regresión Lineal
- 3 Regresión Polinómica**
- 4 Regresión Múltiple

Regresión Polinómica: Modelo Matemático

- La regresión polinómica modela la relación entre la variable dependiente (y) y la independiente (x) como un polinomio de grado n .
- **Modelo Polinómico:**

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_n x^n$$

- $\hat{y}$: Valor predicho.
- x : Variable independiente.
- $\theta_0, \theta_1, \dots, \theta_n$: Coeficientes del modelo.
- n : Grado del polinomio.
- El modelo polinómico se construye transformando la variable independiente en términos polinómicos ($x^2, x^3, \dots, x^n$).

Regresión Polinómica: Objetivo de Ajuste

- La regresión polinómica minimiza el error cuadrático medio (MSE) entre los valores reales y predichos:

$$MSE = \frac{1}{n} \sum_{i=1}^n (\text{price}_i - \text{predictions}_i)^2$$

- **Proceso:**

- Transformar x en términos polinómicos.
- Resolver como un problema de regresión lineal múltiple usando las características transformadas.

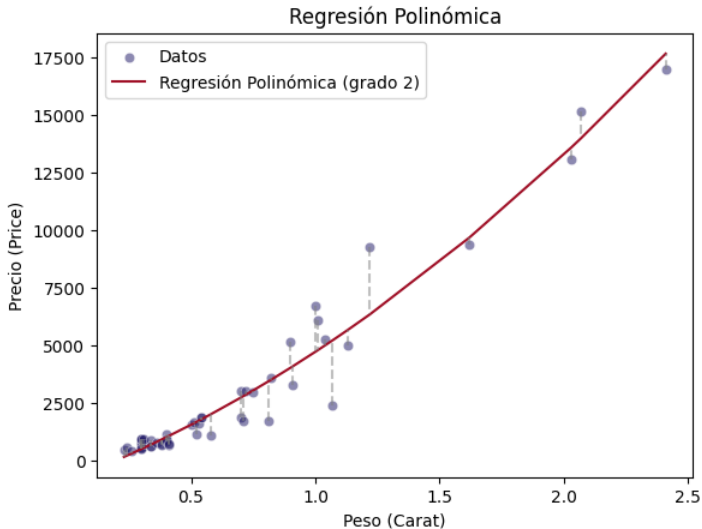
- **Ventaja:**

- Captura relaciones no lineales entre las variables.

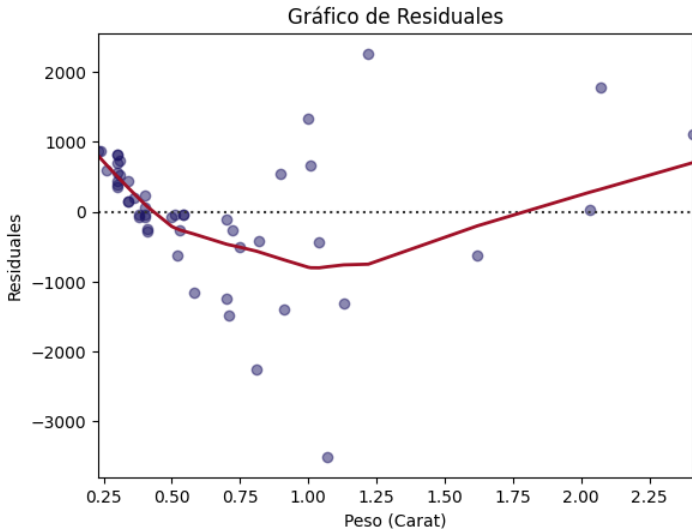
- **Desafío:**

- Grados muy altos (n) pueden llevar al sobreajuste.

Regresión Polinómica



Regresión Polinómica



Regresión Polinómica

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

X = datos[['carat']].values
y = datos['price'].values

# Crear características polinómicas
grado = 2 # Ajustar al grado deseado del polinomio
poly = PolynomialFeatures(degree=grado, include_bias=True)
X_poly = poly.fit_transform(X)

# Entrenar el modelo de regresión
model_poly = LinearRegression()
model_poly.fit(X_poly, y)

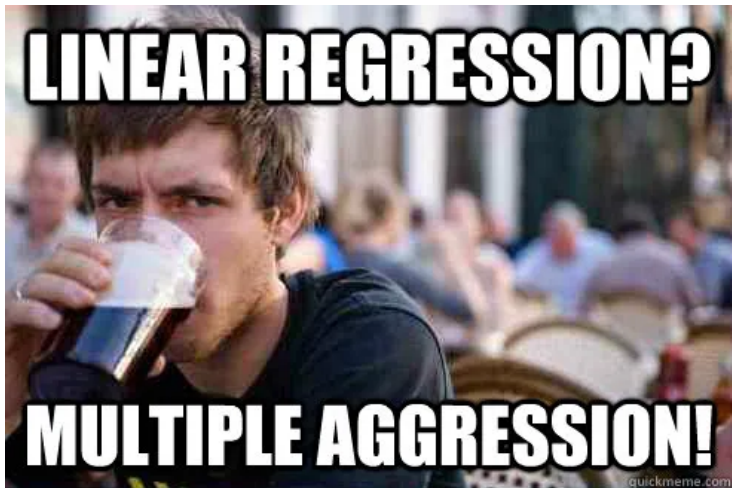
# Predicciones
y_pred = model_poly.predict(X_poly)

# Métricas de evaluación
mse_poly = mean_squared_error(y, y_pred)
rmse_poly = np.sqrt(mse_poly)
r2_poly = r2_score(y, y_pred)
```


Contenido

- 1 Dataset Diamonds
- 2 Regresión Lineal
- 3 Regresión Polinómica
- 4 Regresión Múltiple

Regresión Múltiple



¿Qué es la Regresión Lineal Múltiple?

- **Definición:** Extensión de la regresión simple con múltiples variables independientes.

- **Ecuación:**

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- Ejemplo en diamonds:
 - Predecir price usando carat, cut, clarity y volume.

Preparación de Datos: Creación de Nueva Variable

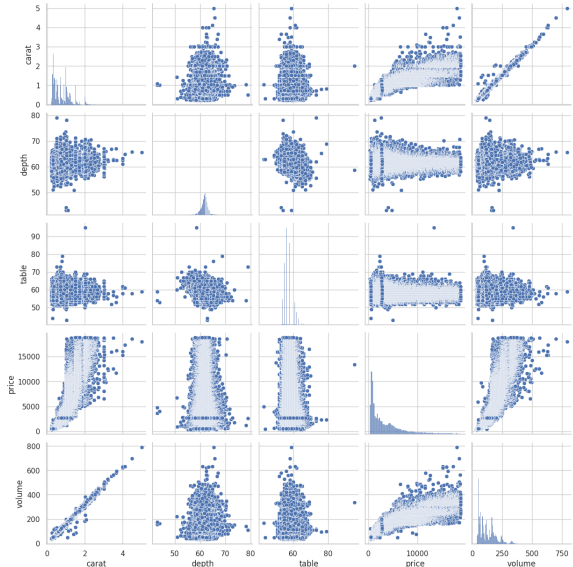
- Creación de una nueva variable volume, que representa el volumen del diamante calculado como el producto de las dimensiones x, y y z.
- Eliminación de las columnas x, y y z, ya que ahora están representadas en volume.

```
diamond_data['volume'] = diamond_data['x'] * diamond_data['y'] * diamond_data['z']  
diamond_data.drop(['x', 'y', 'z'], axis=1, inplace=True)  
diamond_data.head()
```

Gráfico de Pares (Pairplot)

Exploración visual de relaciones entre las variables numéricas del conjunto de datos.

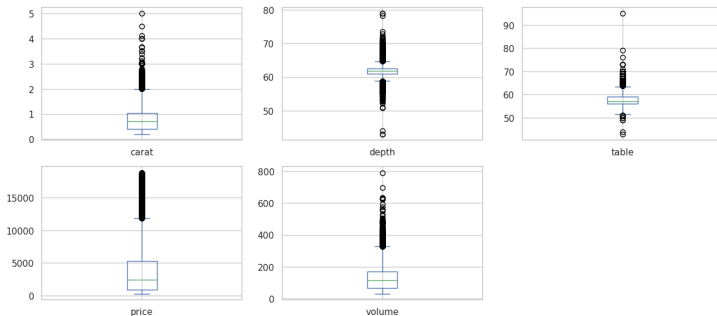
```
sns.pairplot(diamond_data)  
plt.show()
```



Diagramas de Caja (Boxplots)

- Distribución de las variables numéricas, identificando valores atípicos mediante gráficos de caja.

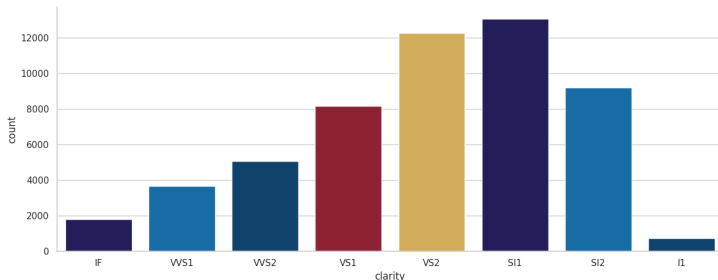
```
diamond_data.plot(kind='box', figsize=(15, 10), subplots=True, layout=(3, 3))  
plt.show()
```



Conteo por Categorías de Claridad (Clarity)

- Distribución de frecuencias para las categorías en la variable clarity.

```
categories = diamond_data['clarity'].nunique()  
sns.catplot(x='clarity', data=diamond_data, hue='clarity', kind='count',  
            aspect=2.5, palette=est_palette[:categories])  
plt.show()
```



Codificación de Variables Categóricas

- Uso de codificación con 'LabelEncoder' para transformar variables categóricas en valores numéricos.
- Transformación aplicada a las columnas cut, color y clarity.
- Esto permite el uso de estas variables en modelos de regresión y otros algoritmos.

```
from sklearn import preprocessing
label_encoder = preprocessing.LabelEncoder()

diamond_data['cut'] = label_encoder.fit_transform(diamond_data['cut'])
diamond_data['color'] = label_encoder.fit_transform(diamond_data['color'])
diamond_data['clarity'] = label_encoder.fit_transform(diamond_data['clarity'])
diamond_data.head()
```


Codificación de Variables Categóricas

	carat	cut	color	clarity	depth	table	price	volume
0	0.230000	2	1	3	61.500000	55.000000	326.000000	38.202030
1	0.210000	3	1	2	59.800000	61.000000	326.000000	34.505856
2	0.230000	1	1	4	61.776399	57.373301	327.000000	38.076885
3	0.290000	3	5	5	62.400000	58.000000	334.000000	46.724580
4	0.310000	1	6	3	63.300000	58.000000	335.000000	51.917250
5	0.240000	4	6	7	62.800000	57.000000	336.000000	38.693952
6	0.240000	4	5	6	62.300000	57.000000	336.000000	38.830870
7	0.260000	4	4	2	61.900000	55.000000	337.000000	42.321081
8	0.220000	0	1	5	61.776399	61.000000	337.000000	36.425214
9	0.230000	4	4	4	59.400000	61.000000	338.000000	38.718000
10	0.300000	1	6	2	64.000000	55.000000	339.000000	49.658700
11	0.230000	2	6	4	62.800000	56.000000	340.000000	37.704420
12	0.220000	3	2	2	60.400000	61.000000	342.000000	34.715136
13	0.310000	2	6	3	62.200000	54.000000	344.000000	51.515745
14	0.200000	3	1	3	60.200000	62.000000	345.000000	32.262375

Entrenamiento del Modelo de Regresión Lineal Múltiple

- Separación del conjunto de datos en variables independientes (X) y dependiente (y).
- Entrenamiento del modelo de regresión lineal múltiple utilizando las variables transformadas.
- La variable price se utiliza como objetivo.

```
X = diamond_data.drop(columns=['price'])  
y = diamond_data['price']
```

```
model_multi = LinearRegression()  
model_multi.fit(X, y)
```

Entrenamiento del Modelo de Regresión Lineal

```
from sklearn.linear_model import LinearRegression

model_multi = LinearRegression()
model_multi.fit(X_multi_transformed, y)
```

Entrenamiento del Modelo de Regresión Lineal

- **Objetivo:** Entrenar un modelo de regresión lineal múltiple con las características preprocesadas.
- `LinearRegression()`: Crea un modelo de regresión lineal.
- `fit()`:
 - Toma como entrada las características (X) y la variable objetivo (y).
 - Ajusta los coeficientes ($\theta_0, \theta_1, \dots, \theta_n$) minimizando el error cuadrático medio (MSE).
- **Resultado:** Modelo entrenado para predecir price usando carat, cut, color, volume y clarity.

Evaluación del Modelo de Regresión Lineal Múltiple

- Predicción de los valores de y (precio) utilizando el modelo entrenado.
- Cálculo de métricas clave para evaluar el rendimiento del modelo:
 - **RMSE**: Raíz del error cuadrático medio, mide la precisión de las predicciones.
 - **R²**: Coeficiente de determinación, mide qué tan bien el modelo explica la variabilidad de los datos.

```
y_pred = model_multi.predict(X)
print("RMSE: {}".format(np.sqrt(mean_squared_error(y, (y_pred)))))
print("R2 : {}".format(np.sqrt(r2_score(y, (y_pred)))))
```

Validación del Modelo con División de Datos

- División del conjunto de datos en entrenamiento ($X_{\text{train}}, y_{\text{train}}$) y prueba ($X_{\text{test}}, y_{\text{test}}$).
- Entrenamiento del modelo en el conjunto de entrenamiento y evaluación en el conjunto de prueba.
- Métricas calculadas: **RMSE** y **R²**.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,  
                                                    random_state=42)
```

```
model_multi = LinearRegression()  
model_multi.fit(X_train, y_train)
```

```
y_pred = model_multi.predict(X_test)
```

```
print(f"RMSE: {np.sqrt(mean_squared_error((y_test), (y_pred)))}")  
print(f"R2   {(2_score((y_test), (y_pred)))}")
```

Regresión Múltiple

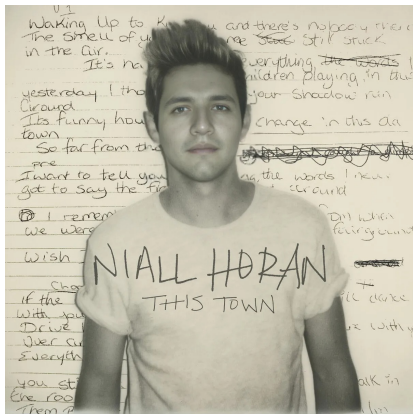


Figura 1: Regresión Univariada



Figura 2: Regresión Multivariada